

Página de Análisis de Datos Genéticos de Antropología Molecular

Alessandro López-Hernández

2026-01-04

Table of contents

1	Bienvenida	4
1.1	Instalación de R y RStudio	4
2	R para principiantes	6
3	Vectores	9
4	Matrices	10
5	Data frames	12
6	Listas	14
7	Trabajos en el Laboratorio I	16
7.1	Descarga de datos desde GenBank	16
8	Alineamiento de secuencias	20
9	Ejercicio	22
10	Análisis de Distancias Genéticas	23
11	Algoritmos de construcción de árboles filogenéticos	27
12	Ejercicio 2:	30
13	Referencias	31
14	¿Qué pasa si solo tengo frecuencias de haplogrupos?	32
15	Evidencia de uso de frecuencias de haplogrupos como proxy de distancia genética	33
16	Recomendaciones del tratamiento de tus datos	35
17	Análisis de las frecuencias de haplogrupos mitocondriales	36
17.1	Test exacto de Fisher para evaluar la asociación entre frecuencias de haplogrupos y factores como la lengua, la cultura o la geografía	36

17.2	Análisis de distancias genéticas a partir de frecuencias de haplogrupos mitocondriales	38
17.3	Árbol de distancias genéticas a partir de la matriz de distancias	44
17.4	PCA	48
18	Summary	59
	References	60

1 Bienvenida

En esta sección, aprenderemos a usar R para trabajar con datos de secuencias de ADN mitocondrial (mtDNA). R es un lenguaje de programación y un entorno de software para análisis estadístico y gráficos. Es ampliamente utilizado en la comunidad científica, especialmente en biología y genética, debido a su capacidad para manejar grandes conjuntos de datos y realizar análisis complejos.

1.1 Instalación de R y RStudio

Para comenzar, necesitamos instalar R y RStudio. R es el lenguaje de programación, mientras que RStudio es un entorno de desarrollo integrado (IDE) que facilita el uso de R. 1. **Instalar R:** Puedes descargar R desde el sitio web del [Proyecto R](#). Sigue las instrucciones para tu sistema operativo (Windows, macOS o Linux). 2. **Instalar RStudio:** Después de instalar R, puedes descargar RStudio desde el sitio web de [RStudio](#). Elige la versión gratuita (RStudio Desktop) y sigue las instrucciones de instalación.

Una vez que hayas instalado R y RStudio, abre RStudio para comenzar a trabajar con tus datos de secuencias de ADN mitocondrial. En la próxima sección, aprenderemos cómo cargar y manipular estos datos en R.

¡Bienvenidos al **Laboratorio de Antropología Molecular** de la Facultad de Ciencias, UNAM!.



Figure 1.1: Miembros del Laboratorio de AM en Teotihuacán. Créditos a Alessandro López-Hernandez, 2023.

2 R para principiantes

Nota: esta sección está diseñada para estudiantes que no tienen experiencia previa en programación, en especial a R. Si ya tienes experiencia en programación, puedes saltar esta sección.

En esta sección, aprenderemos a usar R para trabajar con datos de secuencias de ADN mitocondrial (mtDNA). R es un lenguaje de programación y un entorno de software para análisis estadístico y gráficos. Es ampliamente utilizado en la comunidad científica, especialmente en biología y genética, debido a su capacidad para manejar grandes conjuntos de datos y realizar análisis complejos.

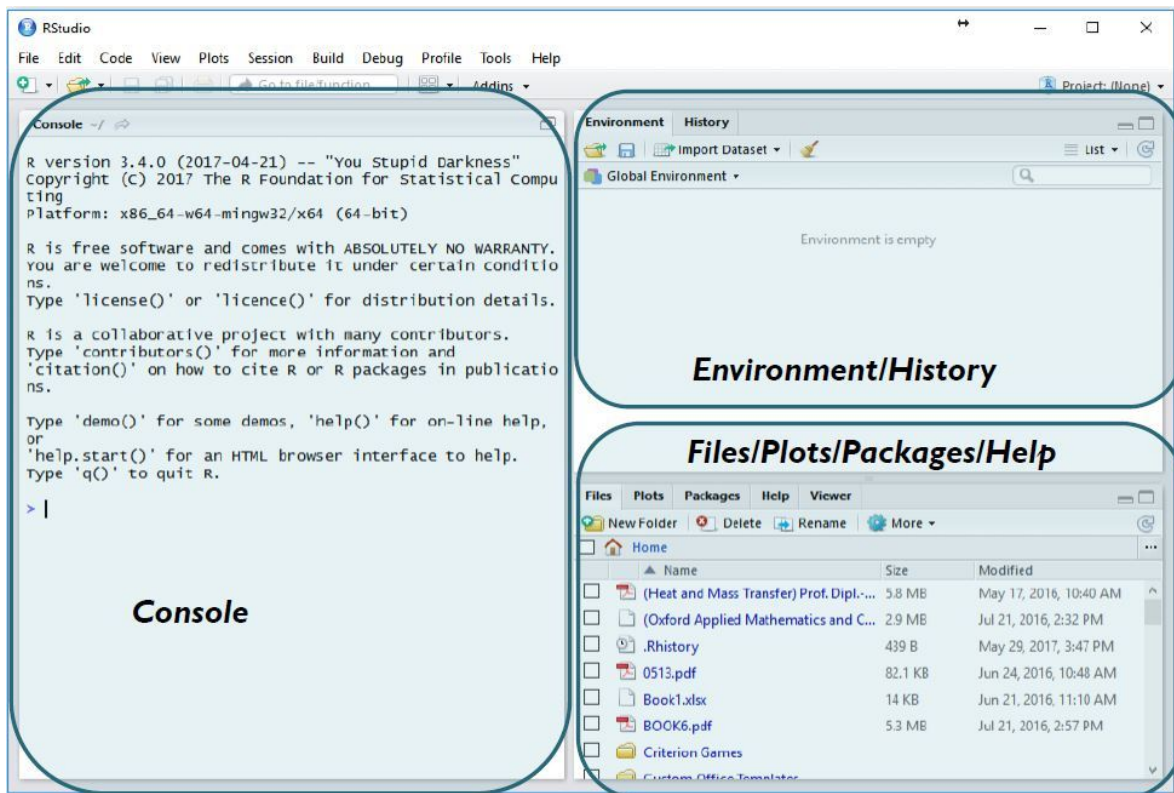


Figure 2.1: Interfaz de RStudio, un entorno de desarrollo integrado para R. Créditos a “Geeks-ForGeeks”

Arriba pueden observar lo que es una interfaz de RStudio. La ventana de la izquierda es la consola. La consola es donde se ejecutan las instrucciones que escribimos en el script. Por ejemplo, si queremos cargar un paquete, tenemos que escribir `library(nombre_del_paquete)` en el script y luego ejecutar esa línea de código para que se cargue el paquete en la consola.

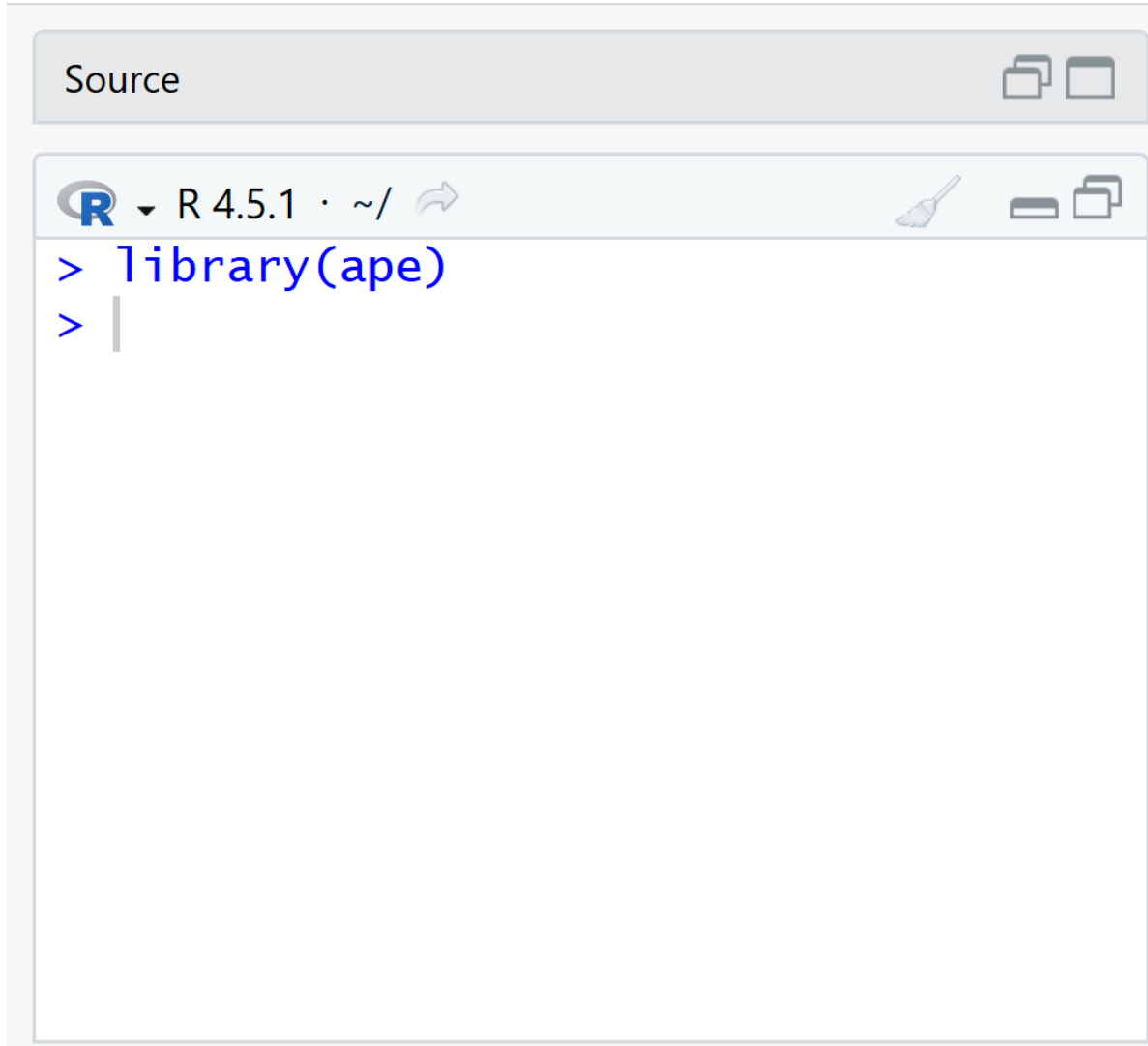


Figure 2.2: Ejecución de una instrucción en RStudio.

Como notarás, no se ejecuta nada más. !Y eso está bien!. No prestemos por ahora atención sobre los mensajes que aparecen en la consola. Lo importante es que se ejecute la instrucción que escribimos en el script. **En este caso, lo que hicimos fue cargar el paquete ape, que es un paquete muy útil para trabajar con datos de secuencias de ADN.**

A partir de esta sección, ya no verás imágenes de la consola de RStudio ejecutándose, sino que ahora verás el código tal cuál debe escribirse en tu consola. Esto es para que puedas copiar y pegar el código directamente en tu consola de RStudio y así practicar lo que estás aprendiendo.

Lo siguiente que haremos es generar datos para que R pueda leerlos. Los datos en R se clasifican en diferentes tipos: *vectores*, *matrices*, *data frames* y *listas*.

3 Vectores

Los vectores son la forma más básica de datos en R. Un vector es *una secuencia de elementos del mismo tipo*. Por ejemplo, un vector de números enteros, un vector de caracteres, etc. Para crear un vector en R, usamos la función `c()`, que significa “combine” o “concatenate”. Por ejemplo, para crear un vector de números enteros del 1 al 5, escribimos:

```
mi_vector <- c(1, 2, 3, 4, 5)
```

En este caso, `mi_vector` es un vector que contiene los números del 1 al 5. Podemos acceder a los elementos de un vector usando índices. Por ejemplo, para acceder al tercer elemento de `mi_vector`, escribimos:

```
mi_vector[3]
```

```
[1] 3
```

Esto nos devolverá el número 3, que es el tercer elemento del vector. También podemos realizar operaciones con vectores. Por ejemplo, si queremos sumar 2 a cada elemento del vector, escribimos:

```
mi_vector + 2
```

```
[1] 3 4 5 6 7
```

Esto nos devolverá un nuevo vector con los valores 3, 4, 5, 6 y 7.

Nota: los vectores son muy importantes en R, especialmente cuando trabajamos con datos de secuencias de ADN, ya que podemos organizar las secuencias en un vector para facilitar su análisis. Por ejemplo:

```
secuencias <- c("ATCG", "GCTA", "TACG", "CGTA")
```

En este caso, `secuencias` es un vector que contiene las secuencias de ADN. Cada elemento del vector es una secuencia de ADN representada como una cadena de caracteres. Podemos acceder a cada secuencia usando índices, por ejemplo:

4 Matrices

Las matrices son una extensión de los vectores. Una matriz es *una colección de elementos del mismo tipo organizados en filas y columnas*. Para crear una matriz en R, usamos la función `matrix()`. Por ejemplo, para crear una matriz de 2 filas y 3 columnas con los números del 1 al 6, escribimos:

```
mi_matriz <- matrix(1:6, nrow = 2, ncol = 3)
```

En este caso, `mi_matriz` es una matriz que contiene los números del 1 al 6 organizados en 2 filas y 3 columnas. Podemos acceder a los elementos de una matriz usando índices de fila y columna. Por ejemplo, para acceder al elemento en la primera fila y segunda columna, escribimos:

```
mi_matriz[1, 2]
```

```
[1] 3
```

Esto nos devolverá el número 4, que es el elemento en la primera fila y segunda columna de la matriz. También podemos realizar operaciones con matrices. Por ejemplo, si queremos sumar 2 a cada elemento de la matriz, escribimos:

```
mi_matriz + 2
```

```
      [,1] [,2] [,3]  
[1,]    3    5    7  
[2,]    4    6    8
```

Esto nos devolverá una nueva matriz con los valores 3, 4, 5, 6, 7 y 8.

Nota: las matrices son muy importantes en R, especialmente cuando trabajamos con datos de secuencias de ADN, ya que podemos organizar las secuencias en una matriz para facilitar su análisis. Por ejemplo:

```
secuencias_matriz <- matrix(c("ATCG", "GCTA", "TACG", "CGTA"), nrow = 2, ncol = 2)
secuencias_matriz
```

```
      [,1] [,2]
[1,] "ATCG" "TACG"
[2,] "GCTA" "CGTA"
```

En este caso, `secuencias_matriz` es una matriz que contiene las secuencias de ADN organizadas en 2 filas y 2 columnas. Cada elemento de la matriz es una secuencia de ADN representada como una cadena de caracteres.

5 Data frames

Los data frames son una estructura de datos muy común en R. Un data frame es *una tabla de datos que puede contener diferentes tipos de variables*. Para crear un data frame en R, usamos la función `data.frame()`. Por ejemplo, para crear un data frame con información sobre secuencias de ADN, escribimos:

```
secuencias_df <- data.frame(  
  id = c("seq1", "seq2", "seq3", "seq4"),  
  secuencia = c("ATCG", "GCTA", "TACG", "CGTA"),  
  longitud = c(4, 4, 4, 4)  
)
```

En este caso, `secuencias_df` es un data frame que contiene información sobre las secuencias de ADN. La primera columna `id` contiene los identificadores de las secuencias, la segunda columna `secuencia` contiene las secuencias de ADN y la tercera columna `longitud` contiene la longitud de cada secuencia. Podemos acceder a las columnas de un data frame usando el operador `$`. Por ejemplo, para acceder a la columna `secuencia`, escribimos:

```
secuencias_df$secuencia
```

```
[1] "ATCG" "GCTA" "TACG" "CGTA"
```

Esto nos devolverá un vector con las secuencias de ADN. También podemos acceder a las filas de un data frame usando índices. Por ejemplo, para acceder a la primera fila del data frame, escribimos: `zwqsecuencias_df[1, ]` Esto nos devolverá la primera fila del data frame, que contiene la información sobre la secuencia `seq1`. También podemos realizar operaciones con data frames. Por ejemplo, si queremos agregar una nueva columna que contenga el número de A's en cada secuencia, escribimos: `zwqsecuencias_df$num_A <- sapply(secuencias_df$secuencia, function(x) sum(str_count(x, "A")))` Esto nos agregará una nueva columna `num_A` al data frame que contiene el número de A's en cada secuencia de ADN.

Nota: los data frames son muy importantes en R, especialmente cuando trabajamos con datos de secuencias de ADN, ya que podemos organizar la información sobre las secuencias en un data frame para facilitar su análisis. Por ejemplo, podemos

agregar más columnas al data frame para incluir información adicional sobre las secuencias, como la especie a la que pertenecen, la ubicación geográfica donde se encontraron, etc. Esto nos permitirá realizar análisis más complejos y obtener insights más profundos sobre nuestras secuencias de ADN.

6 Listas

Las listas son una estructura de datos muy flexible en R. Una lista es *una colección de elementos que pueden ser de diferentes tipos*. Para crear una lista en R, usamos la función `list()`. Por ejemplo, para crear una lista que contenga información sobre secuencias de ADN, escribimos:

```
secuencias_lista <- list(  
  id = c("seq1", "seq2", "seq3", "seq4"),  
  secuencia = c("ATCG", "GCTA", "TACG", "CGTA"),  
  longitud = c(4, 4, 4, 4)  
)
```

En este caso, `secuencias_lista` es una lista que contiene información sobre las secuencias de ADN. La lista tiene tres elementos: `id`, `secuencia` y `longitud`. Cada elemento de la lista es un vector que contiene la información correspondiente. Podemos acceder a los elementos de una lista usando el operador `$`. Por ejemplo, para acceder al elemento `secuencia`, escribimos:

```
secuencias_lista$secuencia
```

```
[1] "ATCG" "GCTA" "TACG" "CGTA"
```

Esto nos devolverá un vector con las secuencias de ADN. También podemos acceder a los elementos de una lista usando índices. Por ejemplo, para acceder al primer elemento de la lista, escribimos:

```
secuencias_lista[[1]]
```

```
[1] "seq1" "seq2" "seq3" "seq4"
```

Esto nos devolverá el vector `id` que contiene los identificadores de las secuencias. Las listas son muy útiles cuando queremos almacenar información que no necesariamente tiene la misma estructura, como por ejemplo, una lista de secuencias de ADN donde cada secuencia puede tener una longitud diferente. Por ejemplo, podemos crear una lista de secuencias de ADN donde cada secuencia es un vector de caracteres de diferente longitud:

```
secuencias_lista <- list(  
  seq1 = c("A", "T", "C", "G"),  
  seq2 = c("G", "C", "T", "A"),  
  seq3 = c("T", "A", "C"),  
  seq4 = c("C", "G")  
)
```

En este caso, **secuencias_lista** es una lista que contiene cuatro elementos, cada uno de los cuales es un vector de caracteres que representa una secuencia de ADN. Cada secuencia tiene una longitud diferente, lo que demuestra la flexibilidad de las listas en R.

Con eso concluye esta sección introductoria sobre R para principiantes. En las siguientes secciones, aprenderemos a usar R para realizar análisis más complejos con nuestros datos de secuencias de ADN mitocondrial.

7 Trabajos en el Laboratorio I

En el laboratorio de Antropología Molecular, es común trabajar con datos de secuencias de ADN mitocondrial (mtDNA). En esta sección, aprenderemos a descargar datos desde GenBank y a leerlos en R para su posterior análisis.

Nota: Asegúrate de tener listos tus archivos de secuencias en formato FASTA antes de comenzar. Si no los tienes, puedes continuar siguiendo las instrucciones que daré a continuación.

7.1 Descarga de datos desde GenBank

GenBank es una base de datos pública de secuencias de ADN mantenida por el NCBI [National Center for Biotechnology Information; Sayers et al. (2022)]. Para descargar datos de GenBank, puedes utilizar la función `rentrez` en R o hacerlo manualmente a través de la página web del NCBI. Para descargar los datos manualmente, naveguen a la página de NCBI y seleccionen el formato FASTA como se muestra a continuación:

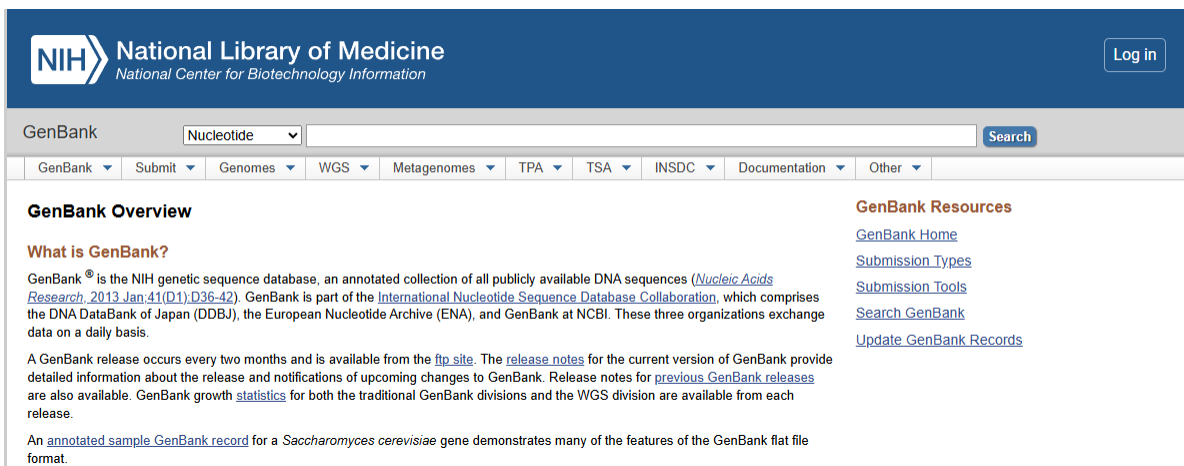


Figure 7.1: Página de inicio de NCBI. Las opciones por default se muestran en la imagen

Lo primero que debemos hacer es buscar una secuencia de interés. Para esto, podemos utilizar el número de acceso (accession number) o el nombre del organismo. En este caso, decidí usar solo las secuencias de mtDNA de la región control, que es una región comúnmente utilizada en

estudios de genética de poblaciones. Para esto, busqué “Human mitochondrial DNA control region” en la barra de búsqueda y seleccioné las secuencias que me interesaban.

Nucleotide

Human mitochondrial DNA control region

Create alertAdvanced

Summary200 per pageSort by Default orderSend to:

Items: 1 to 200 of 11359

<< First < Prev Page 1 of 57 Next > Last >>

Filters activated: Sequence length from 1 to 400. Clear all

☐

[Human mitochondrial DNA control region..genotype AK1110](#)

1. 377 bp linear DNA
Accession: U37731.1 GI: 1022848
[PubMed](#) [Taxonomy](#)
[GenBank](#) [FASTA](#) [Graphics](#)

☐

[Human mitochondrial DNA control region..genotype AK27](#)

2. 377 bp linear DNA
Accession: U37730.1 GI: 1022847
[PubMed](#) [Taxonomy](#)
[GenBank](#) [FASTA](#) [Graphics](#)

☐

[Human mitochondrial DNA control region..genotype AW222](#)

3. 377 bp linear DNA
Accession: U37733.1 GI: 1022850
[PubMed](#) [Taxonomy](#)
[GenBank](#) [FASTA](#) [Graphics](#)

☐

[Human mitochondrial DNA control region..genotype AW218](#)

4. 377 bp linear DNA
Accession: U37732.1 GI: 1022849
[PubMed](#) [Taxonomy](#)
[GenBank](#) [FASTA](#) [Graphics](#)

☐

[Human mitochondrial DNA control region..genotype HK3992](#)

5. 377 bp linear DNA
Accession: U37737.1 GI: 1022854
[PubMed](#) [Taxonomy](#)
[GenBank](#) [FASTA](#) [Graphics](#)

Figure 7.2: Captura de pantalla de la búsqueda en NCBI. En la barra de búsqueda se muestra “HVR1 mtDNA”

Una vez que tenga identificadas las secuencias de interés, puedo descargarlas directamente, sin embargo, para este ejercicio, usaremos el paquete `ape` para descargar las secuencias directamente desde R, lo que nos permitirá automatizar el proceso y evitar errores manuales. Para ello, debo identificar el número de acceso de la secuencia que quiero descargar. En este caso, usaré la secuencia con el número de acceso “U37731.1”, que corresponde a una secuencia de mtDNA humana de la región control.

```
options(warn = -1) # oculta warnings

# Cargamos el paquete "ape"
library(ape)
# Descargamos la secuencia de interés utilizando su número de acceso
secuencia_1 <- read.GenBank("U37731.1")
# Visualizamos la secuencia descargada
print(secuencia_1)
```

1 DNA sequence in binary format stored in a list.

Sequence length: 377

Label:
U37731.1

Base composition:

a	c	g	t
0.322	0.324	0.128	0.226

(Total: 377 bases)

Sin embargo, es necesario que se descarguen al menos un número de secuencias para que el código funcione correctamente. Para esto, podemos descargar varias secuencias utilizando sus números de acceso. Por ejemplo, podemos descargar las siguientes secuencias:

```
# Descargamos varias secuencias utilizando sus números de acceso
secuencias <- read.GenBank(c("U37731.1", "U37730.1", "U37733.1", "U37737.1", "U37752.1",
                             "U37751.1", "U37739.1", "U37743.1"))
# Visualizamos las secuencias descargadas
print(secuencias)
```

8 DNA sequences in binary format stored in a list.

Mean sequence length: 377.125

```
Shortest sequence: 377
Longest sequence: 378
```

```
Labels:
U37731.1
U37730.1
U37733.1
U37737.1
U37752.1
U37751.1
...
```

```
Base composition:
      a      c      g      t
0.320 0.336 0.125 0.219
(Total: 3.02 kb)
```

Sin embargo, es importante mencionar que hay una secuencia que no tiene el mismo tamaño que las demas, lo que puede causar problemas en un analisis posterior. Para evitar esto, es recomendable descargar secuencias que tengan el mismo tamaño o recortarlas para que tengan el mismo tamaño antes de analizarlas o en su defecto, hacer un alineamiento de secuencias para que todas tengan el mismo tamaño. Aquí haremos un alineamiento de secuencias utilizando el paquete `msa` para asegurarnos de que todas las secuencias tengan el mismo tamaño antes de analizarlas.

8 Alineamiento de secuencias

El alineamiento requiere de un paquete adicional llamado `msa`, que se debe de instalar antes de continuar. Para instalarlo, puedes usar el siguiente código:

```
# Instalación de Bioconductor
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")
# Instalación del paquete msa
BiocManager::install("msa")
```

Bioconductor es un proyecto de software de código abierto que proporciona herramientas para el análisis de datos genómicos y biológicos. El paquete `msa` es una herramienta de alineamiento de secuencias que permite alinear múltiples secuencias de ADN, ARN o proteínas. Para alinear las secuencias descargadas, podemos usar la función `msa` del paquete `msa`.

****Nota:** Para que el alineamiento funcione correctamente, el paquete `msa` requiere que las secuencias estén en un formato “AAStringSet” o “DNAStrngSet”. Sin embargo, el paquete `ape` devuelve las secuencias en un formato “DNABin”. Por lo tanto, es necesario transformar las secuencias descargadas en un formato compatible con el paquete `msa` antes de alinearlas. Para esto, podemos usar la función `as.DNABin` del paquete `ape` para transformar las secuencias en un formato compatible con el paquete `msa`:

```
suppressPackageStartupMessages(library(msa))
suppressPackageStartupMessages(library(Biostrings))

# Descargamos las secuencias utilizando sus números de acceso
secuencias_crudas <- read.GenBank(c("U37731.1", "U37730.1", "U37733.1", "U37737.1", "U37752.1",
                                   "U37751.1", "U37739.1", "U37743.1"))

# Este paso es esencial para convertir las secuencias al formato que el paquete msa requiere
secuencias_letras <- as.character(secuencias_crudas) # Transformamos las secuencias en un formato compatible

# Esto convierte c("A", "T", "G") en "ATG" para cada individuo
secuencias_texto <- sapply(secuencias_letras, paste, collapse = "")

# Convertimos el texto limpio al formato que msa exige
```

```

secuencias_biostrings <- DNASTringSet(secuencias_texto)

# Alineamos las secuencias utilizando la función msa
alineamiento <- msa(secuencias_biostrings, method = "Muscle")

# Visualizamos el alineamiento
print(alineamiento)

```

MUSCLE 3.8.31

Call:

```
msa(secuencias_biostrings, method = "Muscle")
```

MsaDNAMultipleAlignment with 8 rows and 378 columns

	aln	names
[1]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGMT-AGGGGTCCCTTGAC	U37733.1
[2]	TTCTTTTATGGGGMAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCATGAC	U37731.1
[3]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCTTGAC	U37730.1
[4]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCTTGAC	U37752.1
[5]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCTTGAC	U37737.1
[6]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCTTGAC	U37739.1
[7]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCTTGAC	U37743.1
[8]	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGATCAGGGGTCCCTTGAC	U37751.1
Con	TTCTTTCATGGGGAAGCAGATTTGGG...CCCCTCAGAT-AGGGGTCCCTTGAC	Consensus

¡Felicidades! Has generado un alineamiento de secuencias utilizando el paquete `msa`. Este alineamiento es esencial para realizar análisis posteriores, como la construcción de árboles filogenéticos o el cálculo de distancias genéticas entre las secuencias. En la próxima sección, aprenderemos a construir un árbol filogenético utilizando el alineamiento generado.

9 Ejercicio

1. Descarga al menos entre 10 a 30 secuencias de de DNA diferentes organismos (pueden ser humanos o de otras especies) utilizando el paquete `ape` o manualmente desde GenBank. **Nota: Asegúrate de que las secuencias descargadas sean de la misma región del ADN para evitar problemas en el análisis posterior. Por ejemplo, puedes descargar secuencias de la región control del mtDNA de diferentes individuos o especies. Además, evita usar secuencias mayores a 1000 pb para no sobrecargar el sistema de tu computadora.**
2. Alinea las secuencias descargadas utilizando el paquete `msa` y visualiza el alineamiento.
3. Compara el alineamiento generado con las secuencias originales para asegurarte de que todas las secuencias tengan el mismo tamaño y que el alineamiento se haya realizado correctamente.
4. Guarda el alineamiento en un archivo FASTA para su posterior análisis. Para ello, necesitas usar la función `writeXStringSet` del paquete `Biostrings` para guardar el alineamiento en un archivo FASTA. Aquí te dejo un ejemplo de cómo hacerlo:

```
# Guardamos el alineamiento en un archivo FASTA
alineamiento_texto <- as(alineamiento, "DNASTringSet")

# El archivo se va a guardar donde sea que tengas tu proyecto de R. Si quieres guardarlo en v
writeXStringSet(alineamiento_texto, filepath = "../Results/alineamiento.fasta")
```

10 Analisis de Distancias Genéticas

Una vez que tenemos nuestro alineamiento de secuencias, podemos calcular las distancias genéticas entre las secuencias para entender las relaciones evolutivas entre ellas. Para esto, podemos usar la función `dist.dna` del paquete `ape`, que calcula las distancias genéticas entre las secuencias alineadas utilizando diferentes modelos de sustitución de nucleótidos. Anteriormente, era necesario transformar el alineamiento al formato “DNABin” para que la función `dist.dna` pueda trabajar con él. Sin embargo, el paquete `msa` devuelve el alineamiento en un formato “MsaDNAMultipleAlignment”, por lo que es necesario transformar el alineamiento al formato “DNABin” antes de calcular las distancias genéticas. Para esto, podemos usar la función `as.DNABin` del paquete `ape` para transformar el alineamiento al formato “DNABin”:

```
# Transformamos el alineamiento al formato "DNABin"
alineamiento_dnabin <- as.DNABin(alineamiento)

# Calculamos las distancias genéticas entre las secuencias utilizando la función dist.dna
distancias_geneticas <- dist.dna(alineamiento_dnabin, model = "K80") # Puedes cambiar el modelo

# Visualizamos las distancias genéticas
print(distancias_geneticas)
```

```
          U37733.1    U37731.1    U37730.1    U37752.1    U37737.1
U37731.1 0.049988834
U37730.1 0.047081849 0.008042973
U37752.1 0.044371795 0.032909975 0.030085109
U37737.1 0.035679771 0.035571725 0.032754816 0.030029993
U37739.1 0.041481614 0.035750892 0.032909975 0.030228985 0.027228865
U37743.1 0.047278780 0.035750892 0.032909975 0.030228985 0.027228865
U37751.1 0.035750892 0.018942587 0.016195327 0.019024034 0.016178692
          U37739.1    U37743.1
U37731.1
U37730.1
U37752.1
U37737.1
U37739.1
U37743.1 0.010782089
U37751.1 0.016261596 0.016261596
```

En este punto, estas funciones te serán conocidas puesto que has trabajado con ellas en el Laboratorio de Antropología Molecular. La distancia genética usada en este modelo es el modelo de Kimura de 2 parámetros (K80), que asume que las sustituciones de nucleótidos pueden ser transiciones (cambio entre purinas o entre pirimidinas) o transversiones (cambio entre purina y pirimidina), y que las tasas de sustitución son diferentes para cada tipo de cambio (Elias and Lagergren 2007).

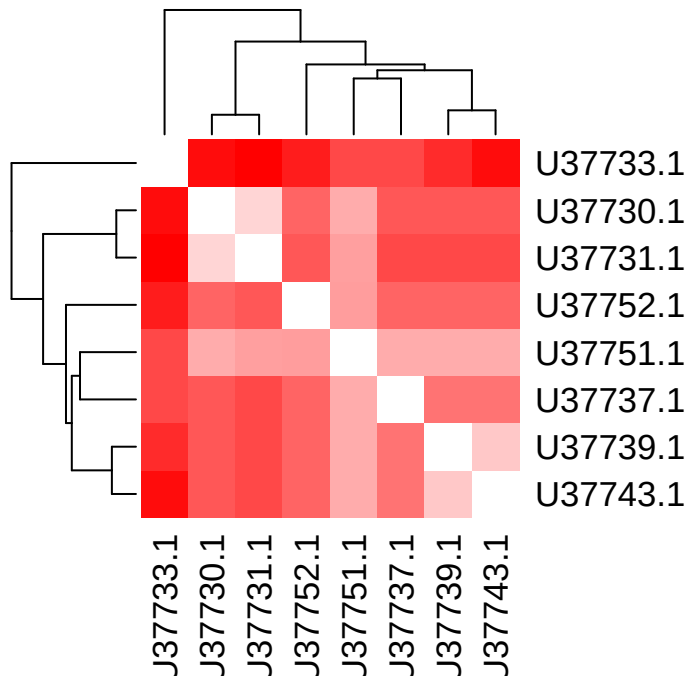


Figure 10.1: Modelo de sustitución de acuerdo a Kimura (1980) o conocido como “Kimura de 2 parámetros”. La probabilidad de transición se define como alfa α mientras que la probabilidad de transversión se define como beta β

Nota: El modelo de Kimura de 2 parámetros es uno de los modelos más simples para calcular distancias genéticas, pero existen otros modelos más complejos que pueden ser utilizados dependiendo del tipo de datos y del análisis que se quiera realizar. Es importante elegir el modelo adecuado para obtener resultados precisos y confiables. Discutirlo con tu tutor(a) es muy importante.

Ahora que tenemos las distancias genéticas, es necesario observar gráficamente las distancias entre secuencias. Podemos hacer esto generando un plot de las distancias genéticas utilizando un plot de calor (heatmap) o un dendrograma. Para generar un heatmap, podemos usar la función `heatmap` del paquete `stats`:

```
# Generamos un heatmap de las distancias genéticas
heatmap(as.matrix(distancias_geneticas), symm = TRUE, col = colorRampPalette(c("white", "red"
```

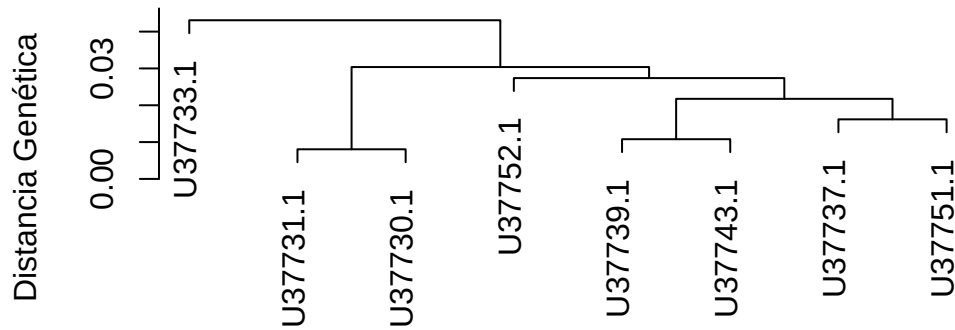


Los colores en el heatmap representan la magnitud de las distancias genéticas entre las secuencias, donde el blanco representa distancias genéticas cercanas a cero (secuencias muy similares) y el rojo representa distancias genéticas mayores (secuencias más diferentes).

Para generar un dendrograma, podemos usar la función `hclust` del paquete `stats`:

```
# Generamos un dendrograma de las distancias genéticas
dendrograma <- hclust(distancias_geneticas, method = "average")
plot(dendrograma, main = "Dendrograma de Distancias Genéticas", xlab = "Secuencias", ylab = "Distancias Genéticas")
```

Dendrograma de Distancias Genéticas



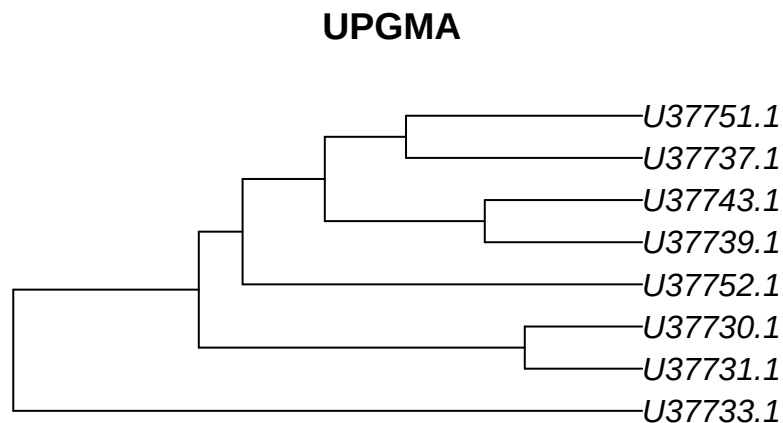
Secuencias
hclust (*, "average")

El dendrograma muestra las relaciones evolutivas entre las secuencias, donde las secuencias que están más cercanas entre sí en el dendrograma tienen distancias genéticas menores, lo que sugiere que están más relacionadas evolutivamente. Las secuencias que están más alejadas entre sí en el dendrograma tienen distancias genéticas mayores, lo que sugiere que están menos relacionadas evolutivamente.

11 Algoritmos de construcción de árboles filogenéticos

Una vez que tenemos las distancias genéticas entre las secuencias, podemos construir un árbol filogenético para visualizar las relaciones evolutivas entre las secuencias. Para esto, podemos usar diferentes algoritmos de construcción de árboles filogenéticos, como el método de UPGMA (Unweighted Pair Group Method with Arithmetic Mean) o el método de Neighbor-Joining. En este caso, usaremos el método de UPGMA para construir el árbol filogenético utilizando la función `upgma` del paquete `phangorn`:

```
# Cargamos el paquete "phangorn" para usar la función upgma
suppressPackageStartupMessages(library(phangorn))
# Construimos el árbol filogenético utilizando el método de UPGMA
tree_upgma <- upgma(distancias_geneticas)
# Visualizamos el árbol filogenético
plot(tree_upgma, main = "UPGMA")
```



Y también podemos usar el método de Neighbor-Joining para construir el árbol filogenético utilizando la función `nj`:

```
# Construimos el árbol filogenético utilizando el método de Neighbor-Joining
tree_nj <- nj(distancias_geneticas)
# Visualizamos el árbol filogenético
plot(tree_nj, main = "Neighbor-Joining")
```

Neighbor-Joining



Hay que recordar que UPGMA asume que las tasas de evolución son constantes a lo largo del tiempo (reloj molecular), lo que puede no ser siempre el caso, mientras que Neighbor-Joining no hace esta suposición y puede ser más adecuado para datos con tasas de evolución variables. Es importante elegir el método adecuado para construir el árbol filogenético dependiendo de las características de los datos y del análisis que se quiera realizar. Discutirlo con tu tutor(a) es muy importante.

Por otra parte, es importante generar un árbol filogenético con soporte estadístico para evaluar la confiabilidad de las relaciones evolutivas entre las secuencias. Para esto, podemos usar el método de bootstrap para generar múltiples réplicas del árbol filogenético y calcular el soporte estadístico para cada nodo del árbol. Para esto, podemos usar la función `bootstrap` del paquete `phangorn`:

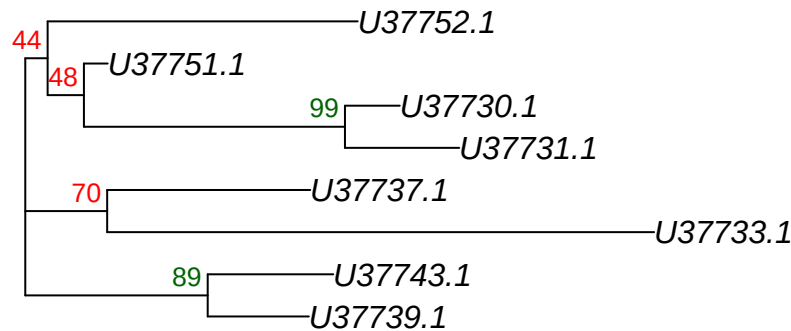
```
fun_nj <- function(x) nj(dist.dna(x, model = "K80"))
# Generamos un árbol filogenético con soporte estadístico utilizando el método de bootstrap
```

```
bs_values <- boot.phylo(tree_nj, alineamiento_dnabin, fun_nj, B = 100)# B es el número de rép
```

```
Running bootstraps:      100 / 100  
Calculating bootstrap values... done.
```

```
# Visualizamos el árbol filogenético con soporte estadístico  
plot(tree_nj, main = "Árbol Final con Soporte de Bootstrap")  
nodelabels(bs_values, adj = c(1.2, -0.5), frame = "n", cex = 0.8,  
            col = ifelse(bs_values > 70, "darkgreen", "red")) # Agregamos los valores de boo
```

Árbol Final con Soporte de Bootstrap



En este código, `boot.phylo` genera múltiples réplicas del árbol filogenético utilizando el método de bootstrap y calcula el soporte estadístico para cada nodo del árbol. Los valores de bootstrap se agregan a los nodos del árbol utilizando la función `nodelabels`, donde los nodos con valores de bootstrap mayores a 70 se colorean en verde oscuro, indicando un soporte estadístico fuerte, mientras que los nodos con valores de bootstrap menores o iguales a 70 se colorean en rojo, indicando un soporte estadístico débil. Es importante interpretar los valores de bootstrap con cautela, ya que un valor de bootstrap alto no garantiza que la relación evolutiva representada por ese nodo sea correcta, pero sí sugiere que esa relación es más confiable que un nodo con un valor de bootstrap bajo.

12 Ejercicio 2:

Recuerda que estos datos deben ser generados a partir de los que descargaste anteriormente. 1. Construye un árbol filogenético utilizando el método de UPGMA y otro utilizando el método de Neighbor-Joining. 2. Compara los árboles filogenéticos generados con los diferentes métodos y discute las diferencias y similitudes entre ellos. 3. Genera un árbol filogenético con soporte estadístico utilizando el método de bootstrap y discute la confiabilidad de las relaciones evolutivas entre las secuencias basándote en los valores de bootstrap obtenidos. 4. Guarda los árboles filogenéticos generados en formato Newick para su posterior análisis. Para esto, puedes usar la función `write.tree` del paquete `ape` para guardar los árboles filogenéticos en formato Newick. Aquí te dejo un ejemplo de cómo hacerlo:

```
# Guardamos el árbol filogenético generado con el método de UPGMA en formato New
write.tree(tree_upgma, file = "../Results/arbol_upgma.newick")
# Guardamos el árbol filogenético generado con el método de Neighbor-Joining en formato Newi
write.tree(tree_nj, file = "../Results/arbol_nj.newick")
```

13 Referencias

14 ¿Qué pasa si solo tengo frecuencias de haplogrupos?

Las frecuencias de los haplogrupos pueden ser útiles para algunos análisis. En genética de poblaciones, llevan usándose desde la década de los 1920, cuando los primeros estudios sobre genética aplicado a ganado bovino resultó ser útil para la industria de la carne. Sin embargo, cuentan con ciertas limitantes.

En Antropología Molecular, se ha asociado la proporción de los haplogrupos con la proporción de ancestría de una población. Por ejemplo, Lorenz and Smith (1996) y Smith et al. (1999) demostraron estadísticamente la relación que existe entre la proporción de haplogrupos con la asociación lingüística, cultural y geográfica de las poblaciones estudiadas.

Sin embargo, esta asociación no es tan simple como parece. En primer lugar, la proporción de haplogrupos no siempre refleja la proporción de ancestría de una población. Esto se debe a que los haplogrupos pueden ser compartidos entre diferentes poblaciones debido a la migración, el mestizaje o la deriva genética. Por ejemplo, un haplogrupo común en una población puede ser raro en otra población debido a la migración o al mestizaje. Además, la proporción de haplogrupos puede ser influenciada por factores como la selección natural, la deriva genética o la mutación, lo que puede distorsionar la relación entre la proporción de haplogrupos y la proporción de ancestría.

15 Evidencia de uso de frecuencias de haplogrupos como proxy de distancia genética

Existen tres puntos a favor del uso de las frecuencias mitocondriales como marcador de distancia genética: 1. **Tasa de mutación alta:** El mtDNA tiene una tasa de mutación mucho más alta que el DNA nuclear (de 10 a 100 veces más alta, Brown, George Jr, and Wilson (1979)), lo que permite detectar diferencias genéticas entre poblaciones en un período de tiempo relativamente corto. 2. **Herencia materna:** El mtDNA se hereda exclusivamente por vía materna, lo que significa que no se mezcla con el DNA de otros individuos a través de la reproducción sexual. Esto permite rastrear linajes maternos y estudiar la historia evolutiva de las poblaciones (Wallace 2007). 3. **Sensibilidad a eventos demográficos:** Las frecuencias de haplogrupos pueden ser sensibles a eventos demográficos como migraciones, cuellos de botella y expansiones poblacionales, lo que puede proporcionar información sobre la historia evolutiva de las poblaciones (Cavalli-Sforza, Menozzi, and Piazza 1994).

Si bien existen estas ventajas son importantes, también es cierto que el uso de las frecuencias de los haplogrupos presenta limitaciones significativas. Por otra parte, el estudio del mtDNA en poblaciones humanas nunca antes estudiadas ha demostrado ser útil para entender la historia de las poblaciones indígenas mexicanas. Por ejemplo, Andrés et al. (2014) encontró que las poblaciones indígenas mexicanas tienen una alta diversidad de haplogrupos mitocondriales, lo que sugiere una historia evolutiva compleja y una gran diversidad genética. Además, Gonzalez-Oliver et al. (2017) encontró que las poblaciones indígenas mexicanas tienen una alta proporción de haplogrupos mitocondriales específicos de América, lo que sugiere una historia evolutiva única para estas poblaciones.

En esta sección, analizaremos los resultados obtenidos a partir de las frecuencias de los haplogrupos mitocondriales de las poblaciones indígenas mexicanas. Para ello, se utilizaron los datos obtenidos por López-Hernandez (2019), quien analizó los haplogrupos mitocondriales de la población teenek que habita la Huasteca Potosina, México.

16 Recomendaciones del tratamiento de tus datos

Independientemente de si estos son datos de haplogrupos mitocondriales o haplotipos de la región control, es importante tener en cuenta las siguientes recomendaciones para el tratamiento de tus datos:

1. **Tipo de datos:** Tu tabla de datos (*data frame* en R) debe contener una columna con los nombres de las poblaciones (*pop* en el *data frame* que se usará en el ejemplo), una serie de columnas nombradas con los haplogrupos o haplotipos a usar (*A*, *B*, *C*, etc.). Estas columnas tendrán **la frecuencia de cada haplogrupo o haplotipo en cada población**. Es importante que las frecuencias estén expresadas como proporciones (es decir, entre 0 y 1) y no como porcentajes (entre 0 y 100).
2. **Número de poblaciones:** Para obtener resultados confiables, es recomendable tener al menos 5 poblaciones en tu análisis. Esto se debe a que con un número muy bajo de poblaciones, las estimaciones de distancia genética pueden ser inestables y no representativas de la diversidad genética real.
3. **Columnas con datos lingüísticos, culturales o geográficos:** Si deseas analizar la relación entre las frecuencias de haplogrupos y factores como la lengua, la cultura o la geografía, es importante incluir columnas adicionales en tu *data frame* que contengan esta información. Por ejemplo, podrías tener una columna llamada *lengua* que indique el grupo lingüístico al que pertenece cada población, una columna llamada *cultura* que indique el grupo cultural al que pertenece cada población, y una columna llamada *geografia* que indique la región geográfica donde se encuentra cada población. Esto te permitirá realizar análisis estadísticos para evaluar la asociación entre las frecuencias de haplogrupos y estos factores.
4. **Formato de los datos:** Asegúrate de que tus datos estén en un formato adecuado para la lectura en R. En este ejemplo, usaremos un archivo CSV. Si bien es posible usar otros formatos, como Excel o archivos de texto, el formato CSV es ampliamente compatible y fácil de manejar en R.

17 Análisis de las frecuencias de haplogrupos mitocondriales

17.1 Test exacto de Fisher para evaluar la asociación entre frecuencias de haplogrupos y factores como la lengua, la cultura o la geografía

Como primer paso, cargaremos los datos de las frecuencias de los haplogrupos mitocondriales en el análisis genético de la población teenek. Para ello, usaremos la función `read.csv()` de R para leer el archivo CSV que contiene los datos. Asegúrate de que el archivo esté en el mismo directorio que tu script de R o proporciona la ruta completa al archivo.

```
# Cargar los datos de frecuencias de haplogrupos mitocondriales
datos_abs <- read.csv("data/pop_abs.csv", header = TRUE)
# Mostrar las primeras filas del data frame para verificar que se cargó correctamente
head(datos_abs)
```

	pop	A	B	C	D	Otros	fam_ling	epoca	estado
1	p-MBon	5	0	0	0	0	Maya	Prehispanica	Chiapas
2	p-MPal	5	0	3	1	0	Maya	Prehispanica	Chiapas
3	P-MTab	8	0	7	2	0	Maya	Prehispanica	Tabasco
4	p-MRey	3	0	2	0	0	Maya	Prehispanica	Quintana_Roo
5	p-MXcaret	22	1	2	0	1	Maya	Prehispanica	Quintana_Roo
6	p-MXcam	2	0	0	0	0	Maya	Prehispanica	Yucatan

Como observarás, el data frame contiene una columna llamada *pop* con los nombres de las poblaciones y varias columnas con los nombres de los haplogrupos (por ejemplo, *A*, *B*, *C*, etc.) que contienen el **numero de individuos** de cada haplogrupo en cada población. Esto es necesario para el análisis que realizaremos a continuación, pero es importante recordar que para otros análisis, como el cálculo de distancias genéticas, es necesario convertir estos números en frecuencias (proporciones) dividiendo cada conteo por el total de individuos en cada población.

Ahora, haremos un análisis llamado “Test exacto de Fisher”, para evaluar si existe una asociación significativa entre el número de individuos de cada haplogrupo y la lengua, la cultura o la

geografía. Es una primera exploración para evaluar si hay alguna relación entre las poblaciones. Para ello, usaremos la función `fisher.test()` de R. Esta función realiza el test exacto de Fisher para tablas de contingencia, lo que nos permitirá evaluar la asociación entre las frecuencias de los haplogrupos y los factores mencionados:

```
# 1. Agrupar y sumar los conteos absolutos de individuos por familia lingüística
# Reemplaza A, B, C, D con los nombres de las columnas de tus haplogrupos
tabla_agrupada <- aggregate(cbind(A, B, C, D, Otros) ~ fam_ling, data = datos_abs, FUN = sum)

# 2. El test de Fisher pide una matriz numérica pura, así que quitamos
# la primera columna (el texto de las lenguas) y la pasamos a nombres de fila (rownames)
matriz_contingencia <- as.matrix(tabla_agrupada[, -1])
rownames(matriz_contingencia) <- tabla_agrupada$fam_ling

# 3. Verificamos que la tabla tenga sentido biológico antes del test
print(matriz_contingencia)
```

	A	B	C	D	Otros
Desconocido	23	6	5	2	0
Maya	708	140	135	52	5
Otomangue	25	13	7	6	0
Yuto-azteca	577	218	82	53	8

```
# 4. Correr el test exacto de Fisher
# OJO: simulate.p.value = TRUE es VITAL en genética de poblaciones porque
# las tablas mayores a 2x2 hacen que la memoria de R colapse al calcular factoriales exactos
resultado_lengua <- fisher.test(matriz_contingencia, simulate.p.value = TRUE, B = 10000)

print(resultado_lengua)
```

Fisher's Exact Test for Count Data with simulated p-value (based on 10000 replicates)

```
data: matriz_contingencia
p-value = 9.999e-05
alternative hypothesis: two.sided
```

El resultado del test de Fisher nos dará un valor p que nos indicará si existe una asociación significativa entre las frecuencias de los haplogrupos y la familia lingüística. Si el valor p es menor que un umbral comúnmente utilizado (por ejemplo, 0.05), podemos concluir que hay

una asociación significativa entre las frecuencias de los haplogrupos y la familia lingüística. De lo contrario, no podemos rechazar la hipótesis nula de que no hay asociación. En este caso, obtuvimos un valor p de 9.999e-05 (o 0.00009), lo que indica que hay una asociación significativa entre las frecuencias de los haplogrupos y la familia lingüística en las poblaciones estudiadas. Puedes repetir este proceso para evaluar la asociación entre las frecuencias de los haplogrupos y otros factores como la cultura o la geografía, simplemente cambiando la variable de agrupación en el paso 1 (por ejemplo, usando *cultura* o *geografia* en lugar de *fam_ling*).

NOTA: Es importante tener en cuenta que el test de Fisher es sensible a la distribución de los datos y puede no ser adecuado para tablas con frecuencias muy bajas. En casos donde las frecuencias son muy bajas, es posible que sea necesario usar otros métodos estadísticos o agrupar los datos de manera diferente para obtener resultados más confiables.

NOTA 2: Este test es ligeramente diferente al hecho con AMOVA, que se basa en la varianza genética entre y dentro de las poblaciones. El test de Fisher se basa en la asociación entre frecuencias de haplogrupos y factores como la lengua, la cultura o la geografía, mientras que AMOVA evalúa la estructura genética de las poblaciones en función de la varianza genética. Ambos enfoques pueden ser complementarios para entender la historia evolutiva de las poblaciones, pero es importante elegir el método adecuado según el tipo de datos y las preguntas de investigación que se quieran responder.

17.2 Análisis de distancias genéticas a partir de frecuencias de haplogrupos mitocondriales

Lo siguiente es hacer una matriz de distancias genéticas a partir de las frecuencias de los haplogrupos mitocondriales. Para ello, usaremos la función `dist()` de R para calcular las distancias genéticas entre las poblaciones. Antes de calcular las distancias, es importante convertir los conteos absolutos de individuos en frecuencias (proporciones) dividiendo cada conteo por el total de individuos en cada población (esto ya lo hice previamente y lo separé en un archivo llamado `pop_freq.csv`). Luego, podemos usar la función `dist()` para calcular las distancias genéticas entre las poblaciones basándonos en estas frecuencias.

```
# Cargar los datos de frecuencias de haplogrupos mitocondriales
datos_freq <- read.csv("data/pop_freq.csv", header = TRUE)
# Seleccionar solo las columnas de frecuencias numéricas
frecuencias <- datos_freq[, 2:6]
# Asignar los nombres ANTES de calcular
rownames(frecuencias) <- datos_freq$pop
# Calcular la matriz de distancias genéticas
# 'dist' heredará automáticamente los nombres que acabamos de asignar
distancias <- dist(frecuencias, method = "euclidean")
```

```
# Mostrar la matriz de distancias genéticas
head(distancias)
```

```
[1] 0.5565432 0.6717882 0.5659117 0.1845151 0.2100000 1.3385527
```

Como primer acercamiento, guardaremos el resultado en un objeto llamado `distancias`, que es un objeto de clase “dist” en R. Este objeto contiene las distancias genéticas entre cada par de poblaciones basadas en las frecuencias de los haplogrupos mitocondriales. Lo siguiente es obtener la significancia estadística de los valores obtenidos en el análisis anterior. Para ello, volveremos a usar nuestra tabla de conteos absolutos `datos_abs`:

```
# Creamos una matriz varía para guardar los resultados
matriz_conteos <- datos_abs[, 1:6]
rownames(matriz_conteos) <- matriz_conteos[, 1]
matriz_conteos <- matriz_conteos[, -1]
num_pobs <- nrow(matriz_conteos)
mis_p_valores <- matrix(NA, nrow = num_pobs, ncol = num_pobs)
# Ponerle los nombres de las poblaciones
rownames(mis_p_valores) <- rownames(matriz_conteos)
colnames(mis_p_valores) <- rownames(matriz_conteos)
# Hacer un loop para comparar cada población con las demás
# El Bucle Optimizado (Calcula la mitad y hace un espejo)
for (i in 1:num_pobs) {
  # OJO AQUÍ: 'j' empieza en 'i', no en 1.
  # Esto hace que R solo recorra el triángulo superior.
  for (j in i:num_pobs) {

    if (i == j) {
      mis_p_valores[i, j] <- 1.0000
    } else {
      tabla_par <- matriz_conteos[c(i, j), ]
      columnas_con_datos <- sum(colSums(tabla_par) > 0)

      if (columnas_con_datos < 2) {
        p_val_calculado <- 1.0000
      } else {
        test_par <- fisher.test(tabla_par, simulate.p.value = TRUE, B = 2000)
        p_val_calculado <- test_par$p.value
      }
    }
  }

  # AQUÍ ESTÁ LA MAGIA: Guardamos el mismo p-valor en la
```

```

        # coordenada [i,j] (arriba) y en la [j,i] (abajo).
        mis_p_valores[i, j] <- p_val_calculado
        mis_p_valores[j, i] <- p_val_calculado
    }
}
}

```

Una vez obtenida la matriz de p-valores, podemos usarla para evaluar la significancia de las distancias genéticas entre las poblaciones. Por ejemplo, podríamos establecer un umbral de significancia (por ejemplo, 0.05) y marcar en la matriz de distancias cuáles pares de poblaciones tienen una distancia genética significativa según el test de Fisher. Esto nos permitirá identificar qué poblaciones están genéticamente más cercanas o más lejanas entre sí basándonos en las frecuencias de los haplogrupos mitocondriales.

```

# Crear una matriz para marcar las distancias significativas
mat_dist <- as.matrix(distancias)
# Usamos los valores p calculados previamente para marcar las distancias significativas
p_valores <- as.matrix(mis_p_valores)
# Usamos los mismos nombres de filas y columnas para la matriz de p-valores
rownames(p_valores) <- rownames(mat_dist)
colnames(p_valores) <- colnames(mat_dist)
# Generamos una función similar a la de Arlequin
generar_arlequin <- function(distancias, p_val) {
  n <- nrow(distancias)
  # Crear matriz vacía de texto
  res <- matrix("", nrow = n, ncol = n)
  rownames(res) <- rownames(distancias)
  colnames(res) <- colnames(distancias)

  for(i in 1:n) {
    for(j in 1:n) {
      if(i > j) {
        # Triángulo inferior: Mostrar la distancia (4 decimales como Arlequin)
        res[i, j] <- sprintf("%.4f", distancias[i, j])
      } else if(i < j) {
        # Triángulo superior: Mostrar los símbolos basados en el p-valor
        p <- p_val[i, j]
        if(p < 0.01) {
          res[i, j] <- "*" # Altamente significativo
        } else if(p <= 0.05) {
          res[i, j] <- "+" # Significativo
        } else {

```

```

        res[i, j] <- "-" # No significativo
      }
    } else {
      # Diagonal
      res[i, j] <- "0.0000"
    }
  }
}
# Devolver como data.frame para que se imprima bonito en la consola
return(as.data.frame(res))
}

# Visualizamos parcialmente
matriz_arlequin <- generar_arlequin(mat_dist, p_valores)
head(matriz_arlequin)

```

	p-MBon	p-MPal	P-MTab	p-MRey	p-MXcaret	p-MXcam	p-MCop	c-MXca	Chol	Chor
p-MBon	0.0000	-	-	-	-	-	*	-	-	-
p-MPal	0.5565	0.0000	-	-	-	-	+	-	-	-
P-MTab	0.6718	0.1158	0.0000	-	+	-	+	-	-	+
p-MRey	0.5659	0.0819	0.1319	0.0000	-	-	+	-	-	-
p-MXcaret	0.1845	0.3914	0.5061	0.4104	0.0000	-	*	-	-	-
p-MXcam	0.2100	0.6064	0.7152	0.6092	0.3073	0.0000	+	-	-	-

	Itza	Laca	MCam	MQui1	MQui2	MQui3	MYuc1	MYuc2	MYuc3	Poqo	Tenek	Tojo
p-MBon	-	-	-	-	-	-	-	-	-	-	-	*
p-MPal	+	*	-	-	-	-	+	-	-	+	+	*
P-MTab	*	*	+	+	*	+	*	-	+	*	*	*
p-MRey	+	*	-	-	-	-	-	-	-	-	-	*
p-MXcaret	-	+	-	-	-	-	-	-	-	-	+	*
p-MXcam	-	-	-	-	-	-	-	-	-	-	-	-

	Tzel	Tzol	p-Cho	p-Teo	p-OtXal	c-Mix	p-Azt	p-NXal	p-Tet	NCDMX1	NCDMX2
p-MBon	-	-	-	-	-	-	-	-	-	-	-
p-MPal	-	-	-	-	-	-	-	-	-	+	+
P-MTab	-	-	-	-	*	+	*	+	-	*	*
p-MRey	-	-	-	-	+	-	-	-	-	-	-
p-MXcaret	-	*	-	-	*	+	+	-	-	-	*
p-MXcam	-	-	-	-	-	-	-	-	-	-	-

	NGro1	NGro2	NHgo1	NHgo2	NPue1	NPue2	NSLP	NVer1	NVer2	NVer3	NVer4
p-MBon	-	-	-	-	-	-	-	-	-	-	-
p-MPal	*	-	+	-	-	-	-	*	*	*	+
P-MTab	*	*	*	*	*	+	+	*	*	*	*
p-MRey	+	-	-	-	-	-	-	+	*	+	+

p-MXcaret	*	*	-	*	-	-	+	-	*	*	+
p-MXcam	-	-	-	-	-	-	-	-	-	-	-

Y de la misma forma, podemos generar un plot que muestre tanto las distancias genéticas como la significancia de estas. Para ello, usaremos la función `ggplot2` de R para crear un gráfico de dispersión que muestre las distancias genéticas entre las poblaciones, con colores o símbolos que indiquen la significancia de estas distancias según el test de Fisher.

```
library(ggplot2)

# Preparar los datos: Convertimos las matrices a un formato "largo" (tabla)
df_dist <- as.data.frame(as.table(mat_dist))
# Limitamos el número de decimales a 4 para que se vea como Arlequin
df_dist$Freq <- round(df_dist$Freq, 4)
# Asignamos nombres a las columnas para mayor claridad
colnames(df_dist) <- c("Poblacion_A", "Poblacion_B", "Distancia")

df_p <- as.data.frame(as.table(p_valores))
df_dist$P_valor <- df_p$Freq

# Filtrar solo el triángulo inferior de la matriz
# (Para que el gráfico se vea como la clásica escalera de distancias)
indices_inferiores <- lower.tri(mat_dist)
df_plot <- df_dist[as.vector(indices_inferiores), ]

# Crear las categorías de colores basadas en los P-valores
df_plot$Significancia <- cut(df_plot$P_valor,
                             breaks = c(-Inf, 0.01, 0.05, Inf),
                             labels = c("Altamente Significativo (p <= 0.01)",
                                          "Significativo (p <= 0.05)",
                                          "No Significativo (p > 0.05)"))

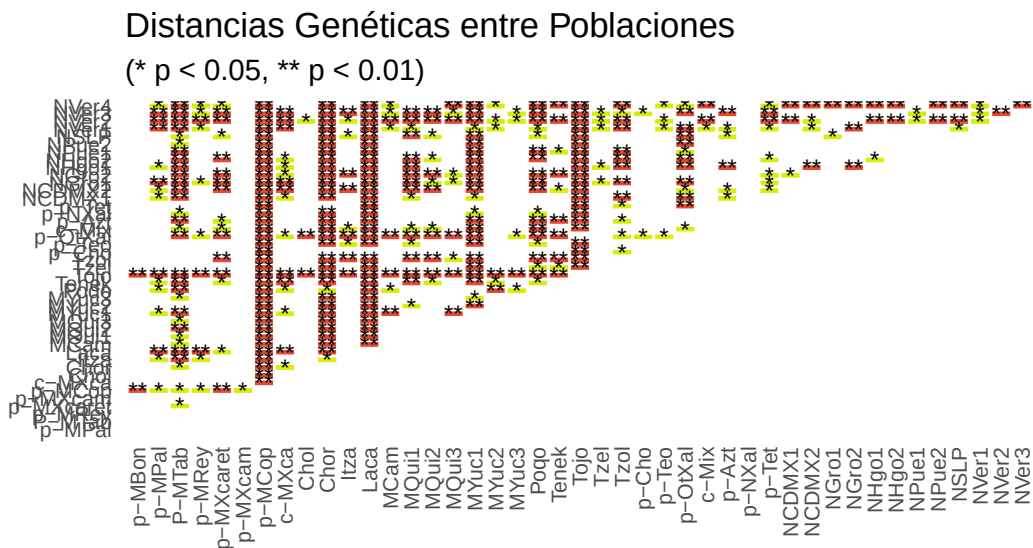
# Asegurar el orden correcto de las poblaciones en los ejes
df_plot$Poblacion_A <- factor(df_plot$Poblacion_A, levels = rownames(mat_dist))
df_plot$Poblacion_B <- factor(df_plot$Poblacion_B, levels = colnames(mat_dist))

# Crear una columna nueva solo con símbolos (estilo Arlequin limpio)
df_plot$Simbolo <- ifelse(df_plot$P_valor <= 0.01, "**",
                          ifelse(df_plot$P_valor <= 0.05, "*", ""))

grafico_distancias <- ggplot(df_plot, aes(x = Poblacion_B, y = Poblacion_A, fill = Significancia)) +
  geom_tile(color = "white", linewidth = 0.5) +
```

```
# Usamos el símbolo en lugar del número largo. Se verá súper limpio.
geom_text(aes(label = Simbolo), size = 3, color = "black") +
scale_fill_manual(values = c("Altamente Significativo (p <= 0.01)" = "#e34a33",
                             "Significativo (p <= 0.05)" = "#d7ee07",
                             "No Significativo (p > 0.05)" = "#ffffff")) +
theme_minimal() +
theme(axis.text.x = element_text(angle = 90, hjust = 1, vjust = 0.5, size = 8), # Letra más
      axis.text.y = element_text(size = 8),
      axis.title = element_blank(),
      panel.grid = element_blank(),
      legend.position = "bottom") +
labs(title = "Distancias Genéticas entre Poblaciones",
     subtitle = "(* p < 0.05, ** p < 0.01)", # Añadimos la leyenda al título
     fill = "Nivel de Significancia:")

print(grafico_distancias)
```



Significancia: ■ Altamente Significativo (p <= 0.01) ■ Significativo (p <= 0.05) ■ No :

```
# Guardar el resultado como una imagen PNG de alta resolución
ggsave("imagenes/mapa_distancias_significancia.png",
      plot = grafico_distancias,
      width = 10, height = 7, dpi = 300)
# Listo
```

¡Personaliza los colores a tu gusto! Es posible también experimentar un poco con el tamaño de los símbolos, el tipo de letra, o incluso agregar etiquetas con los valores de distancia (aunque esto puede hacer que el gráfico se vea muy cargado si hay muchas poblaciones). La clave es encontrar un equilibrio entre la claridad visual y la cantidad de información que deseas mostrar. Por ejemplo, si tienes muchas poblaciones, puede ser mejor usar solo los símbolos para indicar la significancia y omitir los valores numéricos de las distancias para que el gráfico sea más legible. En cambio, si tienes pocas poblaciones, podrías considerar agregar los valores de distancia dentro de cada celda para proporcionar información adicional.

17.3 Árbol de distancias genéticas a partir de la matriz de distancias

Es posible generar un árbol de distancias genéticas a partir de la matriz de distancias que hemos calculado. Este proceso ya lo hemos hecho antes, pero ahora lo haremos específicamente con la matriz de distancias basada en las frecuencias de los haplogrupos mitocondriales. Para ello, usaremos el viejo conocido de este tipo de métodos, el paquete **ape** de R, que nos permite construir árboles filogenéticos a partir de matrices de distancias. En este caso, usaremos el método UPGMA (Unweighted Pair Group Method with Arithmetic Mean) para construir el árbol de distancias genéticas entre las poblaciones basándonos en las frecuencias de los haplogrupos mitocondriales:

```
library(ape)
# Vamos a volver a convertir nuestra matriz a un objeto dist()
dist_obj <- as.dist(mat_dist)
# Generamos un árbol usando el método UPGMA
arbol_upgma <- as.phylo(hclust(dist_obj, method = "average"))
# Graficamos. Usamos el tipo "phylogram" clásico
# Se abrirá en una nueva ventana/pestaña de gráficos en RStudio
# Personaliza el gráfico a tu gusto: colores, tamaños, etc.
# Guardamos el gráfico como una imagen PNG de alta resolución
png(filename = "imagenes/arbol_upgma_publicacion.png",
     width = 2400, height = 2500, res = 300)
# OPCIÓN DE FONDO Y MÁRGENES:
# par(bg = "white") # Aquí puedes cambiar el color de fondo ("transparent", "gray95", etc.)
# par(mar = c(4, 1, 1, 5)) # Ajusta los márgenes: c(Abajo, Izquierda, Arriba, Derecha).
# Si los nombres de la derecha se cortan, sube el último número (el 5).

# Dibujamos el árbol con opciones personalizadas
plot(arbol_upgma,
     type = "phylogram",

     # --- LÍNEAS Y ESTRUCTURA ---
```

```

    edge.width = 1.5,      # Grosor de la línea. 1.5 a 2 es ideal.
    edge.color = "gray20", # Sugerencia: En artículos, el gris oscuro ("gray20") o "black"

    # --- TEXTO DE LAS POBLACIONES ---
    tip.color = "black",   # Color de las etiquetas.
    cex = 0.5,            # ¡OPCIÓN CLAVE PARA EL TAMAÑO! Como tienes muchas poblaciones
    font = 3,             # 1 = Normal, 2 = Negrita, 3 = Cursiva (Estándar científico p

    # --- ESPACIADO Y PRESENTACIÓN ---
    label.offset = 0.001,  # Añade una micro-separación entre la punta de la rama y el t
    no.margin = TRUE,      # Fuerza al árbol a usar todo el espacio de la imagen, borran
    #main = "Árbol Filogenético: UPGMA"
    # Sugerencia: En los artículos científicos ("papers"), las figuras no llevan título den
)
# Modificamos la escala para que haga juego con el nuevo formato
limites <- par("usr")

add.scale.bar(
  x = limites[2] - 0.1,
  y = limites[3] + 0.5,
  length = 0.05,
  cex = 0.6,
  lwd = 1.5,
  col = "black"
)

# Guardamos y cerramos el dispositivo gráfico
invisible(dev.off())

```

Ahora generaremos un NJ (Neighbor-Joining) con la misma matriz de distancias para comparar ambos métodos. El proceso es muy similar, solo cambia el método de clustering que usamos para construir el árbol:

```

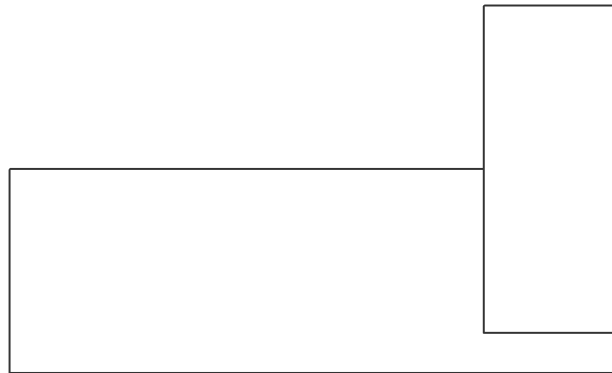
# Generamos un árbol usando el método Neighbor-Joining
arbol_nj <- nj(dist_obj)
# Graficamos el árbol NJ con opciones personalizadas
png(filename = "imagenes/arbol_nj_publicacion.png",
     width = 2400, height = 2500, res = 300)
# par(bg = "white") # Cambia el fondo si quieres
# par(mar = c(4, 1, 1, 5)) # Ajusta los márgenes si es necesario
plot(arbol_nj,
     type = "phylogram",

```

```

    edge.width = 1.5,
    edge.color = "gray20",
    tip.color = "black",
    cex = 0.5,
    font = 3,
    label.offset = 0.001,
    no.margin = TRUE
    #main = "Árbol Filogenético: Neighbor-Joining"
)
# Modificamos la escala para que haga juego con el nuevo formato
limites <- par("usr")
add.scale.bar(
  x = limites[2] - 0.1,
  y = limites[3] + 0.5,
  length = 0.05,
  cex = 0.6,
  lwd = 1.5,
  col = "black"
)
invisible(dev.off())

```



Los resultados de ambos arboles se pueden ver a continuación:



17.4 PCA

El PCA es un análisis multivariado (es decir, que utiliza las varianzas de los datos para reducir en 2 dimensiones la información de los haplogrupos mitocondriales). Para ello usaremos la función `prcomp()` de R, que nos permite realizar un análisis de componentes principales (PCA) a partir de las frecuencias de los haplogrupos mitocondriales. Este análisis nos permitirá

visualizar la relación entre las poblaciones en un espacio bidimensional basado en las frecuencias de los haplogrupos mitocondriales:

```
# OJO: scale. = TRUE es importantísimo para que los haplogrupos raros
# no sean aplastados por los haplogrupos muy comunes.
pca_resultado <- prcomp(frecuencias, center = TRUE, scale. = TRUE)
# Ver un resumen rápido de cuánta varianza explica cada componente
summary(pca_resultado)
```

Importance of components:

	PC1	PC2	PC3	PC4	PC5
Standard deviation	1.3650	1.1526	1.0139	0.8529	0.22924
Proportion of Variance	0.3727	0.2657	0.2056	0.1455	0.01051
Cumulative Proportion	0.3727	0.6384	0.8440	0.9895	1.00000

El resultado del PCA nos dará un resumen de cuánta varianza explica cada componente principal. Ahora, generaremos el gráfico del PCA para visualizar la relación entre las poblaciones en el espacio de los componentes principales:

```
# | message: false
# | warning: false
library(ggrepel)
```

Warning: package 'ggrepel' was built under R version 4.5.3

```
# Crear un data frame para el gráfico del PCA
datos_pca <- as.data.frame(pca_resultado$x[, 1:2])
datos_pca$Poblacion <- rownames(datos_pca)

# Agregar la Familia Lingüística para el color
# (Asumiendo que tienes esta columna en tu base de datos original 'datos_freq')
# Sustituye 'fam_ling' por el nombre real de tu columna.
datos_pca$Familia_Ling <- datos_freq$fam_ling

# Calcular la varianza explicada por cada eje (¡Requisito de publicación!)
varianza <- summary(pca_resultado)$importance[2, 1:2] * 100
etiqueta_x <- sprintf("Componente Principal 1 (0.1f%%)", varianza[1])
etiqueta_y <- sprintf("Componente Principal 2 (0.1f%%)", varianza[2])

# Construir el gráfico con ggplot2
grafico_pca <- ggplot(datos_pca, aes(x = PC1, y = PC2, color = Familia_Ling, label = Poblacion))
```

```

# Añadir líneas de cuadrícula centrales (origen 0,0)
geom_hline(yintercept = 0, linetype = "dashed", color = "gray70") +
geom_vline(xintercept = 0, linetype = "dashed", color = "gray70") +

# Dibujar los puntos de las poblaciones
geom_point(size = 3, alpha = 0.8) +

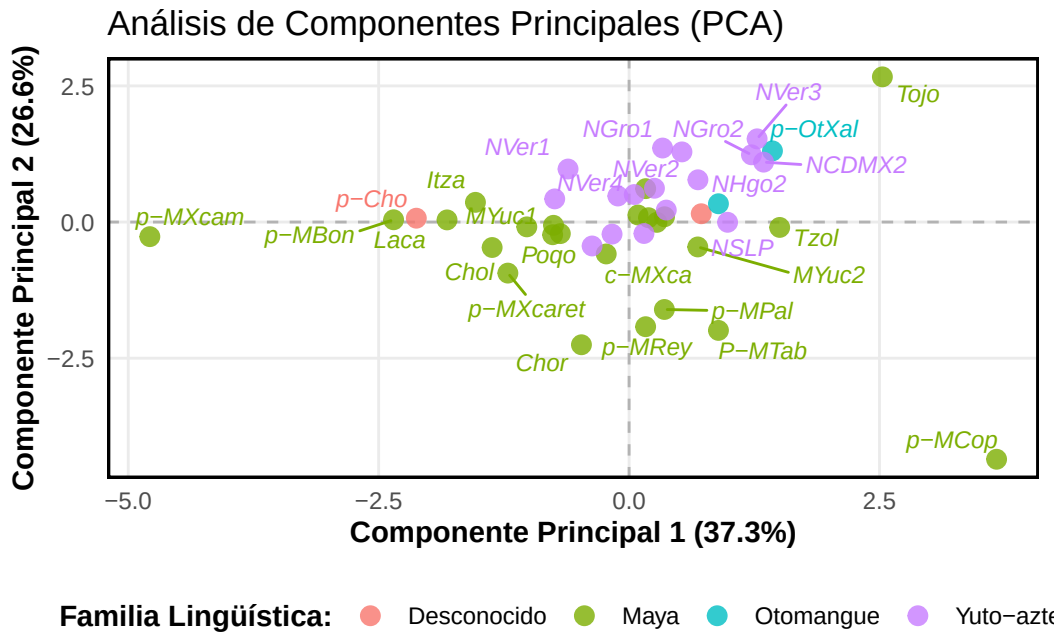
# Agregar los nombres sin que se encimen (ggrepel)
geom_text_repel(size = 3, fontface = "italic", max.overlaps = 15, show.legend = FALSE) +

# Estética limpia (estilo Nature/PLOS)
theme_minimal() +
theme(
  panel.border = element_rect(color = "black", fill = NA, linewidth = 1),
  panel.grid.minor = element_blank(),
  legend.position = "bottom",
  legend.title = element_text(face = "bold"),
  axis.title = element_text(face = "bold")
) +

# Etiquetas de los ejes dinámicas
labs(
  title = "Análisis de Componentes Principales (PCA)",
  x = etiqueta_x,
  y = etiqueta_y,
  color = "Familia Lingüística:"
)

# Mostrar el gráfico en R
print(grafico_pca)

```



```
# Guardar el gráfico del PCA como una imagen PNG de alta resolución
ggsave("imagenes/pca_haplogrupos.png",
  plot = grafico_pca,
  width = 10, height = 7, dpi = 300)
```

Este grafico nos muestra que las poblaciones usadas no se agrupan claramente por familia lingüística. Sin embargo, todavía podemos hacer más exploraciones. A continuación, haremos un PCA-Biplot, que es una forma de visualizar tanto las poblaciones como los haplogrupos en el mismo espacio de componentes principales. Esto nos permitirá ver cómo se relacionan los haplogrupos con las poblaciones en el espacio del PCA:

```
# Crear un biplot del PCA
#Usamos la libreria factoextra para un biplot más elegante
#install.packages("factoextra")
library(factoextra)
# Crear el Biplot
grafico_biplot <- fviz_pca_biplot(pca_resultado,

  # --- CONFIGURACIÓN DE LAS POBLACIONES (PUNTOS) ---
  geom.ind = c("point", "text"), # Mostrar el punto y el nombre de la población
  col.ind = datos_freq$fam_ling, # Colorear puntos por Familia Lingüística
  pointshape = 19,               # Forma del punto (círculo sólido)
```

```

    pointsize = 2.5,                # Tamaño del punto

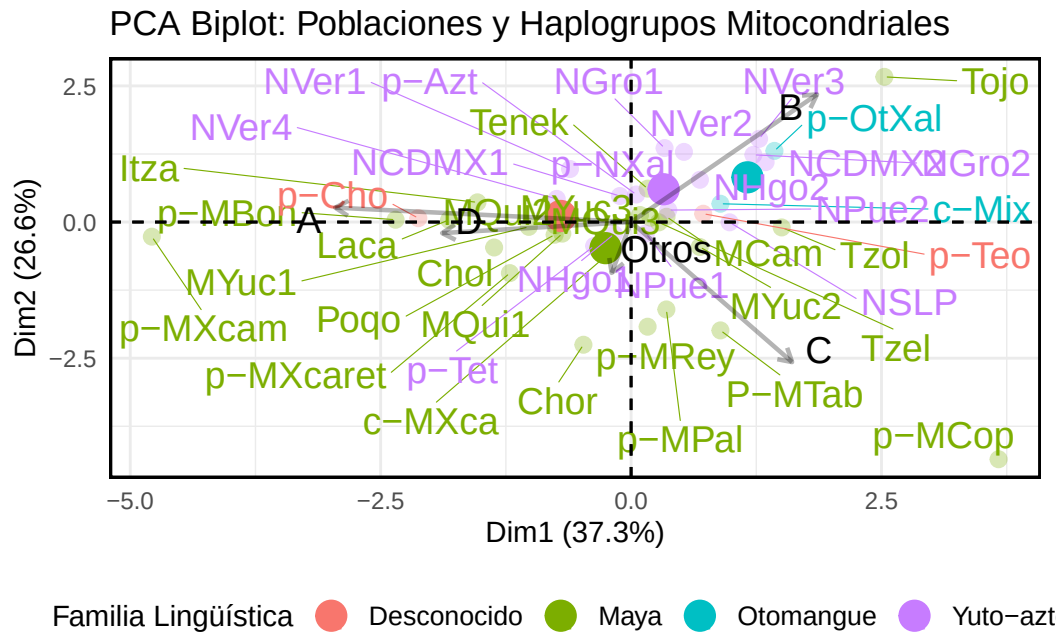
    # --- CONFIGURACIÓN DE LOS HAPLOGRUPOS (FLECHAS) ---
    col.var = "black",              # Las flechas de los haplogrupos en negro para
    arrowsize = 0.8,               # Grosor de la flecha
    labelsize = 5,                 # Tamaño de la letra del haplogrupo (A, B, C,
    alpha = 0.3,                   # Transparencia de los puntos

    # --- ESTÉTICA GENERAL ---
    repel = TRUE,                  # ¡MAGIA! Evita que los nombres se encimen
    legend.title = "Familia Lingüística",
    title = "PCA Biplot: Poblaciones y Haplogrupos Mitocondriales",

    # Opcional: Agregar elipses de confianza alrededor de las familias lingüísticas
    # addEllipses = TRUE, ellipse.level = 0.95
) +
# Como factoextra usa ggplot2 de fondo, podemos seguir sumando capas de personalización:
theme_minimal() +
theme(
  panel.border = element_rect(color = "black", fill = NA, linewidth = 1),
  legend.position = "bottom",
  legend.text = element_text(size = 10)
)

# Mostrar el gráfico
print(grafico_biplot)

```



```
# Exportar en alta calidad
ggsave("imagenes/pca_biplot.png", plot = grafico_biplot, width = 9, height = 7, dpi = 300)
```

Es recomendable que la imagen pueda ser guardada y editada en PDF para que puedas ajustar todos los elementos a tu gusto en un programa de edición de imágenes (como Illustrator o Inkscape) antes de incluirla en tu artículo científico. Para ello, puedes usar la función `ggsave()` con el formato PDF:

```
# Guardar el gráfico del PCA-Biplot como un archivo PDF de alta calidad
ggsave("imagenes/pca_biplot.pdf", plot = grafico_biplot, width = 9, height = 7)
```

Ignoring unknown labels:

```
* fill : "Familia Lingüística"
* linetype : "Familia Lingüística"
* shape : "Familia Lingüística"
```

Sin embargo, este no es el único análisis que podemos hacer. Existe una variante del PCA llamado DAPC (Discriminant Analysis of Principal Components) que es especialmente útil para datos genéticos, ya que maximiza la separación entre grupos (por ejemplo, familias lingüísticas) mientras reduce la dimensionalidad de los datos. Para realizar un DAPC, usaremos el paquete `adegenet` de R, que está diseñado específicamente para análisis genéticos multivariados. Este

análisis nos permitirá visualizar cómo se separan las poblaciones basándonos en las frecuencias de los haplogrupos mitocondriales, teniendo en cuenta la agrupación por familia lingüística:

```
# Instalar y cargar el paquete adegenet si no lo tienes instalado
library(adegenet, quietly = TRUE, warn.conflicts = FALSE)
```

```
Warning: package 'adegenet' was built under R version 4.5.2
```

```
Warning: package 'ade4' was built under R version 4.5.3
```

```
/// adegenet 2.1.11 is loaded //////////////////////////////////
```

```
> overview: '?adegenet'
> tutorials/doc/questions: 'adegenetWeb()'
> bug reports/feature requests: adegenetIssues()
```

```
# Preparar los datos para DAPC
grupos_ling <- as.factor(datos_freq$fam_ling)
# Correr el DAPC de forma automática
# n.pca: Número de componentes a retener. Como tenemos ~5 haplogrupos, retenemos 4.
# n.da: Funciones discriminantes. La regla general es (Número de Grupos - 1).
# Si tenemos 4 familias lingüísticas (Maya, Otomangue, Yuto-Azteca, Desconocido), n.da = 3.
dapc_resultado <- dapc(frecuencias,
                        grp = grupos_ling,
                        n.pca = 4,
                        n.da = 3)
# Ver que haplogrupos pesan más en la discriminación
summary(dapc_resultado)
```

```
$n.dim
[1] 3
```

```
$n.pop
[1] 4
```

```
$assign.prop
[1] 0.7272727
```

```
$assign.per.pop
Desconocido      Maya      Otomangue Yuto-azteca
```

0.000	0.875	0.500	0.625
-------	-------	-------	-------

\$prior.grp.size

Desconocido	Maya	Otomangue	Yuto-azteca
2	24	2	16

\$post.grp.size

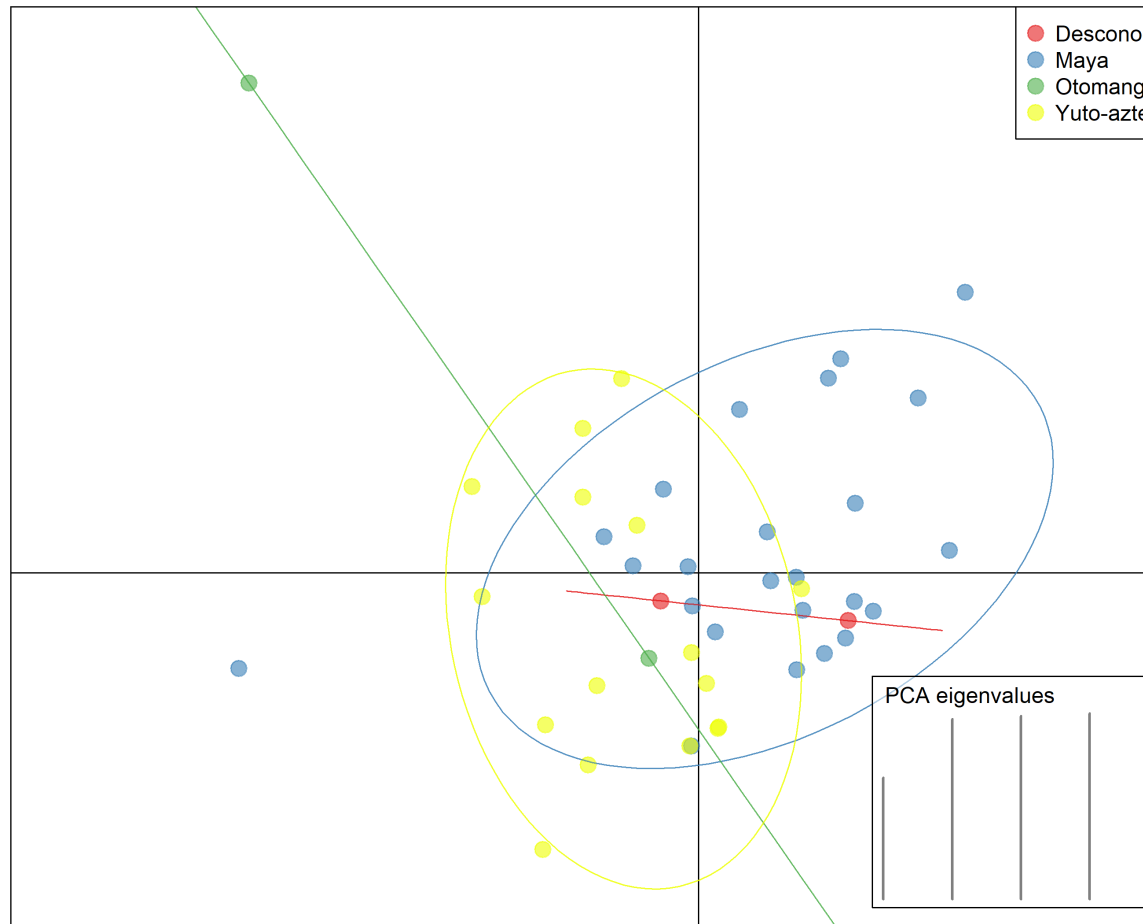
Desconocido	Maya	Otomangue	Yuto-azteca
0	29	1	14

```
# Personalizamos los colores (puedes poner tantos como familias tengas)
mis_colores <- c("#e41a1c", "#377eb8", "#4daf4a", "#eeff00")

png(filename = "imagenes/dapc_scatter.png",
     width = 2700, height = 2100, res = 300)

# R no lo mostrará en tu panel derecho, sino que lo dibujará directo en el archivo.
scatter(dapc_resultado,
        bg = "white",
        pch = 20,
        cstar = 0,
        cell = 2,
        solid = 0.6,
        cex = 2.5,
        col = mis_colores,
        clab = 0,
        leg = TRUE,
        posi.leg = "topright",
        txt.leg = levels(grupos_ling),
        scree.pca = TRUE,
        posi.pca = "bottomright",
        scree.da = FALSE
)

# Cerrar y guardar el archivo (¡CRUCIAL!)
invisible(dev.off())
```



La imagen se verá así:

El análisis DAPC muestra que existe un alto grado de flujo génico o linajes maternos compartidos entre las poblaciones Mayas y Yuto-aztecas, evidenciado por el gran traslape de sus elipses de inercia en el centro del gráfico. Las poblaciones catalogadas como ‘Desconocidas’ presentan un perfil genético intermedio que se inserta dentro de la diversidad de los grupos mayores. Finalmente, destaca la alta divergencia de una de las poblaciones Otomanguenses, la cual se separa drásticamente del clúster central a través del segundo eje discriminante

Por otra parte, la tabla de resultados obtenida por el resumen del DAPC nos muestra qué haplogrupos tienen mayor peso. `$assign.prop` nos indica la proporción de asignación de cada población a cada grupo lingüístico. `$assign.per.pop` nos muestra la proporción de asignación de cada población a cada grupo lingüístico, lo que nos permite ver cómo mientras el grupo Maya tiene una asignación muy alta a su propia familia lingüística, el grupo Yuto-Azteca muestra una asignación más compartida entre varias familias, siendo un total de contribución del 62.5% a su propia familia y el resto repartido entre otras familias. Esto ya había sido descrito antes en López-Hernandez (2019), donde se observaba cómo desde tiempos de la colonia, el náhuatl fue utilizado como lingua franca a lo largo de los tres siglos de la conquista española, obligando a otros pueblos sometidos por los españoles a aprender tanto náhuatl como español

para poder comunicarse con los conquistadores y con otros pueblos indígenas. Esto pudo haber facilitado el flujo génico entre poblaciones de diferentes familias lingüísticas, especialmente entre las poblaciones Yuto-Aztecas y Mayas, que se encuentran en regiones geográficas cercanas y que pudieron haber tenido contacto a lo largo de la historia. Por otra parte, `post.grp.size` es lo que el algoritmo determinó después de analizar los haplogrupos. Para calcular esto, “muestreo aleatoriamente”, ignorando las familias lingüísticas que tu le diste, miró únicamente las frecuencias de los haplogrupos y los asignó genéticamente al grupo al que se parecen más. Si observamos con atención, dos poblaciones anteriormente clasificadas como *Desconocidas* ahora se encuentran en uno de los grupos mayores (ya sea Maya o Yuto-Azteca), lo que sugiere que su perfil genético es más similar a uno de estos grupos que a los otros. Esto podría indicar que estas poblaciones tienen una historia evolutiva más cercana a uno de los grupos mayores, lo que podría ser resultado de eventos históricos como migraciones, mestizaje o intercambio cultural que han influenciado su composición genética a lo largo del tiempo. Para poder observar con mayor detalle cómo se asignaron las poblaciones a los grupos genéticos determinados por el DAPC, podemos crear una tabla de contingencia que muestre la asignación de cada población a cada grupo genético. Esto nos permitirá ver claramente qué poblaciones fueron asignadas a cada grupo y cómo se distribuyen entre las familias lingüísticas:

```
# Crear una tabla de contingencia para mostrar la asignación de poblaciones a grupos genéticos
tabla_asignacion <- table(datos_freq$fam_ling, dapc_resultado$assign)
# Mostrar la tabla de asignación
print(tabla_asignacion)
```

	Desconocido	Maya	Otomangue	Yuto-azteca
Desconocido	0	2	0	0
Maya	0	21	0	3
Otomangue	0	0	1	1
Yuto-azteca	0	6	0	10

```
# Ver los nombres de las poblaciones que "cambiaron" de familia:
poblaciones_cambiadas <- data.frame(
  Poblacion = datos_freq$pop,
  Original = datos_freq$fam_ling,
  Asignado = dapc_resultado$assign
)
# Filtrar solo las que no coinciden
diferentes <- poblaciones_cambiadas[poblaciones_cambiadas$Original != poblaciones_cambiadas$Asignado,]
print(diferentes)
```

Poblacion	Original	Asignado
-----------	----------	----------

18	MYuc2	Maya	Yuto-azteca
21	Tenek	Maya	Yuto-azteca
22	Tojo	Maya	Yuto-azteca
25	p-Cho	Desconocido	Maya
26	p-Teo	Desconocido	Maya
28	c-Mix	Otomangue	Yuto-azteca
29	p-Azt	Yuto-azteca	Maya
31	p-Tet	Yuto-azteca	Maya
36	NHgo1	Yuto-azteca	Maya
38	NPue1	Yuto-azteca	Maya
39	NPue2	Yuto-azteca	Maya
44	NVer4	Yuto-azteca	Maya

El análisis DAPC reveló que la clasificación lingüística no siempre coincide con la estructura genética mitocondrial. La reclasificación de poblaciones ‘Desconocidas’ hacia el grupo Maya, así como la reasignación de múltiples poblaciones Yuto-aztecas como Mayas, sugiere una historia compartida de linajes maternos en el área mesoamericana. Estos resultados indican que las barreras lingüísticas no impidieron el flujo génico entre estos grupos, resultando en una homogeneización parcial de las frecuencias de los haplogrupos A, B, C y D.

18 Summary

In summary, this book has no content whatsoever.

References

(**article?**) {Sayers2022, abstract = {GenBank® (<https://www.ncbi.nlm.nih.gov/genbank/>) is a comprehensive, public database that contains 15.3 trillion base pairs from over 2.5 billion nucleotide sequences for 504 000 formally described species. Recent updates include resources for data from the SARS-CoV-2 virus, including a SARS-CoV-2 landing page, NCBI Datasets, NCBI Virus and the Submission Portal. We also discuss upcoming changes to GI identifiers, a new data management interface for BioProject, and advice for providing contextual metadata in submissions.}, author = {Eric W Sayers and Mark Cavanaugh and Karen Clark and Kim D Pruitt and Conrad L Schoch and Stephen T Sherry and Ilene Karsch-Mizrachi}, doi = {10.1093/nar/gkab1135}, issn = {0305-1048}, issue = {D1}, journal = {Nucleic Acids Research}, month = {1}, pages = {D161-D164}, title = {GenBank}, volume = {50}, url = {<https://doi.org/10.1093/nar/gkab1135>}, year = {2022} }

- Andrés, Moreno-Estrada, Gignoux Christopher R., Fernández-López Juan Carlos, Zakharia Fouad, Sikora Martin, Contreras Alejandra V., Acuña-Alonzo Victor, et al. 2014. “The Genetics of Mexico Recapitulates Native American Substructure and Affects Biomedical Traits.” *Science* 344 (June): 1280–85. <https://doi.org/10.1126/science.1251688>.
- Brown, Wesley M, Matthew George Jr, and Allan C Wilson. 1979. “Rapid Evolution of Animal Mitochondrial DNA.” *Proceedings of the National Academy of Sciences* 76 (4): 1967–71.
- Cavalli-Sforza, Luigi Luca, Paolo Menozzi, and Alberto Piazza. 1994. *The History and Geography of Human Genes*. Princeton university press.
- Elias, Isaac, and Jens Lagergren. 2007. “Fast Computation of Distance Estimators.” *BMC Bioinformatics* 8: 89. <https://doi.org/10.1186/1471-2105-8-89>.
- Faust, Katherine A. 2015. *The Huasteca: Culture, History, and Interregional Exchange*. University of Oklahoma Press.
- Gonzalez-Oliver, Angelica, Ernesto Garfias-Morales, David Glenn Smith, and Mirsha Quinto-Sanchez. 2017. “Mitochondrial DNA Analysis of Mazahua and Otomi Indigenous Populations from Estado de Mexico Suggests a Distant Common Ancestry.” *Human Biology* 89 (July): 195–216.
- Lópe-Hernandez, H Alessandro. 2019. “Análisis de Marcadores Genéticos Del DNA Mitochondrialen Indígenas Teenek Que Habitan La Huasteca Potosina,méxico.” PhD thesis, Ciudad de México, México: Universidad Nacional Autónoma de México.
- Lorenz, Joseph G, and David Glenn Smith. 1996. “Distribution of Four Founding mtDNA Haplogroups Among Native North Americans.” *American Journal of Physical Anthropology: The Official Publication of the American Association of Physical Anthropologists* 101 (3): 307–23.

- Sayers, Eric W, Mark Cavanaugh, Karen Clark, Kim D Pruitt, Conrad L Schoch, Stephen T Sherry, and Ilene Karsch-Mizrachi. 2022. “GenBank.” *Nucleic Acids Research* 50 (January): D161–64. <https://doi.org/10.1093/nar/gkab1135>.
- Smith, David Glenn, Ripan S Malhi, Jason Eshleman, Joseph G Lorenz, and Frederika A Kaestle. 1999. “Distribution of mtDNA Haplogroup x Among Native North Americans.” *American Journal of Physical Anthropology: The Official Publication of the American Association of Physical Anthropologists* 110 (3): 271–84.
- Wallace, Douglas C. 2007. “Why Do We Still Have a Maternally Inherited Mitochondrial DNA? Insights from Evolutionary Medicine.” *Annual Review of Biochemistry* 76: 781–821. <https://doi.org/https://doi.org/10.1146/annurev.biochem.76.081205.150955>.